Lista de Exercícios — Aula 13: Design Patterns com C#

Exercício 1 — Singleton Básico

Crie uma classe chamada `ConfiguracaoSistema` utilizando o padrão Singleton.

Requisitos:

- A classe deve possuir:
  - Nome da aplicação
  - Versão do sistema
- O construtor deve ser privado.
- Deve existir apenas uma instância da classe.
- Teste criando múltiplas variáveis e verifique se todas apontam para o mesmo objeto.

Exercício 2 — Singleton Thread Safe

Implemente um Singleton seguro para aplicações multithread.

Requisitos:

- Utilize `lock`.
- Crie múltiplas threads acessando a instância.
- Verifique se apenas um objeto é criado.

Exercício 3 — Factory de Veículos

Crie uma fábrica de veículos.

Requisitos:

- Interface `IVeiculo`
- Classes:
  - `Carro`
  - `Moto`
  - `Caminhao`
- Método:

CriarVeiculo(string tipo)

Objetivo:

A fábrica deve retornar o objeto correto sem o código cliente conhecer as implementações.

Exercício 4 — Factory de Pagamentos

Crie um sistema de pagamentos usando Factory.

Métodos de pagamento:

- Pix
- Cartão
- Boleto

Requisitos:

- Cada classe deve implementar:

void ProcessarPagamento();

- A fábrica decide qual pagamento criar.

Exercício 5 — Observer de Notificações

Implemente um sistema de notificações.

Cenário:

Um canal do YouTube possui inscritos.

Requisitos:

- Interface `IObserver`
- Classe `Canal`
- Métodos:
    - `Inscrever()`
    - `Remover()`
    - `Notificar()`

Objetivo:

Todos os usuários inscritos devem receber mensagens automaticamente.

Exercício 6 — Observer em Estoque

Crie um sistema de monitoramento de estoque.

Cenário:

Quando um produto voltar ao estoque:

- Clientes cadastrados recebem aviso.

Requisitos:

- Produto observado
- Lista de observadores
- Atualização automática

Exercício 7 — MVC Básico

Crie uma estrutura simples usando MVC.

Requisitos:

Model:

Produto

Controller:

ProdutoController

View:

- Exibir dados do produto no console.

Objetivo:

Separar responsabilidades corretamente.

Exercício 8 — MVC de Cadastro de Usuários

Desenvolva um mini sistema MVC.

Funcionalidades:

- Cadastrar usuário
- Listar usuários
- Exibir detalhes

Estrutura:

- Model → Usuario
- Controller → UsuarioController
- View → Console

Exercício 9 — Integração de Patterns

Crie um sistema combinando padrões.

Cenário:

Sistema de e-commerce.

Utilize:

- Singleton → Configuração do sistema
- Factory → Meios de pagamento
- Observer → Notificações de pedidos
- MVC → Organização da aplicação

Objetivo:

Integrar todos os padrões em um único projeto.

Exercício 10 — Desafio Avançado

Desenvolva um sistema de delivery.

Requisitos:

Singleton

- Gerenciar conexão do banco.

Factory

- Criar diferentes entregadores:
    - Moto
    - Bicicleta
    - Drone

Observer

- Cliente recebe atualizações:
    - Pedido saiu
    - Pedido chegou

MVC

- Organização completa do sistema.

Extras:

- Tratamento de erros
- Logs
- Simulação de pedidos